

КОНЦЕПТ ИСПОЛЬЗОВАНИЯ: ЭКРАНЫ РОБОТА И ДОСТУП К НИМ

Документ: docs/RUN_AND_ACCESS.md **Версия:** 1.2 · **Дата:** 2026-10 **Связанные документы:** [MAIN_SCREEN.md](#) · [GUI.md](#) · [SERVICE_CONSOLE.md](#) · [SERIAL_PROTOCOL.md](#) · [SOFTWARE_ARCHITECTURE.md](#)

Документ отвечает на вопросы: **какие экраны есть у робота, где какой показывается, как устроен сервер и какие у него соединения, что такое PIN-код отсека и как он устанавливается, и кто чем пользуется** в смене и на соревнованиях.

Главное в одной строке: **основной экран — статично на роботе** (там же вводят PIN, и отсек открывается на самом роботе), а **инженерный пульт — удалённо** с ноутбука или планшета. Сервер борта лишь отдаёт страницы.

1. Два экрана — два уровня доступа

Экран	Файл и адрес	Где показывается	Кому	Что показывает
Основной экран робота	main.html, адрес /	статично на дисплее робота: страница всегда открыта, данные обновляются каждые 300 мс	оператор смены, получатель груза, судьи	крупно: 4 двигателя (угол, об/мин, температура, состояние), заряд АКБ в процентах, PIN-код и состояние грузового отсека, режим и связь
Инженерный пульт	index.html, адреса /index.html и /console	удалённо: ноутбук или планшет инженера в сети робота	инженер-наладчик	карта SLAM, миссия, программа доставки, телеметрия модулей, сервисное окно (замок, ручные команды, статистика), журнал

PIN-код вводят **на самом роботе** — на основном экране его дисплея; с этого же экрана открывается и закрывается грузовой отсек борта (сервер выполняет команду замка, см. §3). Удалённо пульт видит то же состояние замка и тот же журнал доступа.

Логика разделения:

- на основном экране **нет** опасных действий — только состояние и открытие отсека по PIN-коду (это нужно и оператору, и получателю груза);
- всё, что меняет поведение робота (ручные команды модулям, режим «стенд», сброс статистики, программы доставки), вынесено на инженерный пульт;
- оба экрана отдаёт **один процесс** — gui/backend.py, поэтому они всегда видят одни и те же данные и одно состояние замка.

2. Как это устроено: один процесс, один порт

Весь «сервер» — это **один процесс Python на борту робота:**

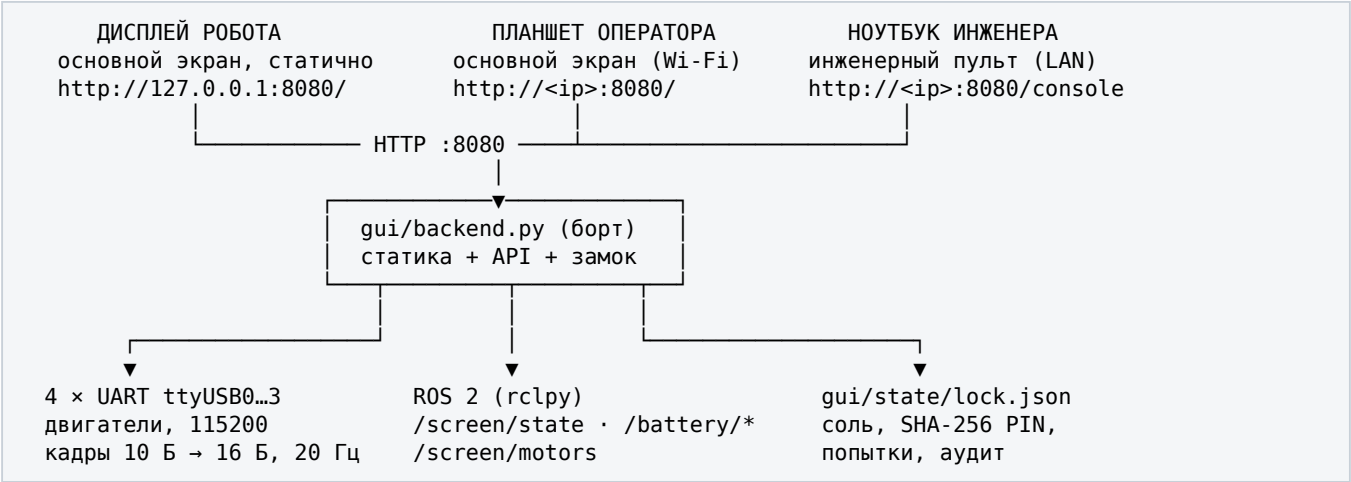
```
python3 gui/backend.py --port 8080          # или короткий синоним: python3 gui/serve.py
```

Он делает три вещи:

1. **отдаёт страницы** — обычные HTML/CSS/JS из папки gui/ (никакой сборки, Node.js и интернета на роботе не нужно — только Python 3 и браузер);
2. **держит JSON API** — /api/state, /api/lock/*, /api/audit, /api/health;

3. **ведёт замок** — проверяет PIN, считает попытки, пишет журнал доступа.

Слушает 0.0.0.0:8080, то есть отвечает и на самом роботе (127.0.0.1), и всем в сети (http://<ip-робота>:8080). Клиенты — только браузеры: браузер на дисплее робота (основной экран), ноутбук или планшет инженера (пульт, удалённо). **Сервер сам ничего не открывает и не запускает** — он только отвечает; показ страницы на дисплее робота обеспечивает отдельный сервис (§4.1). Больше ни одного открытого порта у робота для экранов не требуется.



2.1. Что и куда подключается

Направление	Интерфейс	Что передаётся
дисплей робота → борт	HTTP, 127.0.0.1:8080	GET /api/state (300 мс), POST /api/lock/open — здесь вводят PIN, отсеk открывается на роботе
пульт инженера → борт	HTTP по Wi-Fi/LAN, <ip-робота>:8080	GET /api/state, /api/audit, POST /api/lock/* — то же состояние замка и тот же журнал
борт → модули двигателей	4 × UART /dev/ttyUSB0...3, 115200 8N1 (pyserial)	команды 10 Б с CRC16; телеметрия 16 Б на 20 Гц (протокол — docs/SERIAL_PROTOCOL.md)
борт ↔ ROS 2	DDS через rclpy	подписки /screen/state (JSON), /battery/voltage, /battery/current, /screen/motors
борт → замок отсека	команда ОС из окружения	RUS_SLAM_LOCK_CMD (открыть), RUS_SLAM_LOCK_CMD_CLOSE (закрыть): реле, соленоид, GPIO
борт → диск	файл gui/state/lock.json	соль, хеш PIN, счётчик неудач, блокировка, журнал доступа
борт → браузер робота	не обязателен	по умолчанию ничего не открывается ; страницу на дисплей робота выводит deploy/rus-slam-display.service, а флаги --kiosk / --open делают то же разово вручную

Что из этого обязательного: **ничего, кроме самого процесса**, который отдаёт страницы и данные. Дисплей робота, UART, ROS 2 и железо замка — отдельные подключения, и каждое можно включать по мере готовности робота. По умолчанию источник данных — --source sim (модель): экран, API и замок работают без робота, поэтому систему можно показывать на ноутбуке. Переключатели:

```

python3 gui/backend.py --source sim                # модель (по умолчанию)
python3 gui/backend.py --source serial \           # реальные модули
  --ports /dev/ttyUSB0,/dev/ttyUSB1,/dev/ttyUSB2,/dev/ttyUSB3
python3 gui/backend.py --source ros                # бортовые топики ROS 2
python3 gui/backend.py --source auto              # ROS 2, а без него — модель
  
```

Если для выбранного источника нет библиотеки (pyserial для serial, rclpy для ros) или порт не открывается, борт **не падает**: в консоли появляется предупреждение,

модуль/источник переходит в модель.

3. PIN-код грузового отсека: что это и как устанавливается

PIN-код — это цифровой пароль (4-8 цифр), которым открывается грузовой отсек. Он защищает груз от посторонних: по регламенту конкурса за это дают до 5 баллов (критерий «безопасность груза» — «нет доступа посторонним»). Заводское значение — 2580, его печатает --help и /api/health.

3.1. Как PIN хранится

В открытом виде PIN нигде не лежит:

```
gui/state/lock.json
{
  "version": 1,
  "salt": "<32 hex-символа, случайные 16 байт>",
  "pinHash": "SHA-256(salt + \"\":" + PIN)",
  "fails": 0, // неудачных попыток подряд
  "lockUntil": 0, // до какого времени ввод заблокирован
  "audit": [ ... ], // журнал доступа с хеш-цепочкой (последние 200 записей)
  "changedAt": ...
}
```

Файл создаётся сам при первом запуске сервера и лежит в .gitignore (в репозиторий не попадает). Соль у каждого робота своя, поэтому одинаковые PIN на двух роботах дают разные хеши, а по украденному файлу PIN не подобрать перебором таблицы.

3.2. Три способа установить PIN

Способ	Команда	Когда действует
Заводской при первом старте	python3 gui/backend.py --pin 1234	только если файла lock.json ещё нет (или после сброса)
Смена из сервисного окна пульта	кнопка смены PIN либо POST /api/lock/pin с телом {"current": "2580", "new": "1234"}	нужен текущий PIN; новый — 4-8 цифр, повтор должен совпасть
Сброс к заводскому	python3 gui/backend.py --reset-pin	удаляет файл: следующий запуск создаёт его заново с PIN --pin (по умолчанию 2580)

После успешной смены: генерируется новая соль, старый хеш становится недействительным, отсек закрывается, в журнал пишется pin_change.

3.3. Защита от подбора

- **5 неудачных попыток** подряд → блокировка ввода на **30 секунд** (--max-attempts, --lock-sec меняют пороги);
- счётчик попыток и время блокировки **хранит сервер** (в lock.json), а не страница: перезагрузка экрана, планшета или браузера блокировку не снимает;
- каждая попытка — запись в журнал: успех, отказ с остатком попыток, блокировка, смена PIN. Записи связаны хеш-цепочкой: GET /api/audit возвращает integrity.ok = false, если журнал правили руками;
- ограничение честное: интерфейс работает по **HTTP без шифрования**, поэтому сеть робота должна быть локальной и доверенной (цех, стенд). Развитие — HTTPS и отдельный пароль инженера на пульт (см. §5).

3.4. Проверка PIN через API

```
# неверный PIN → 403, остаток попыток
curl -s -X POST http://127.0.0.1:8080/api/lock/open \
  -H 'Content-Type: application/json' -d '{"pin":"1111"}'
# {"ok": false, "reason": "wrong", "attemptsLeft": 4, ...}

# верный PIN → 200, отсек открыт (и команда железу, если задан RUS_SLAM_LOCK_CMD)
curl -s -X POST http://127.0.0.1:8080/api/lock/open \
  -H 'Content-Type: application/json' -d '{"pin":"2580"}'

# смена PIN
curl -s -X POST http://127.0.0.1:8080/api/lock/pin \
  -H 'Content-Type: application/json' -d '{"current":"2580","new":"1234"}'
```

4. Как попадать на экраны

4.1. Основной экран на работе (статичная страница)

Сервер борта окон не открывает. Страницу на дисплее робота показывает отдельный сервис — так она всегда на месте, а сервер можно перезапускать отдельно от показа:

```
# на работе: сервер + статичная страница на дисплее
sudo systemctl enable --now rus-slam-server      # gui/backend.py на 0.0.0.0:8080
sudo systemctl enable --now rus-slam-display      # браузер без рамок на http://127.0.0.1:8080/
```

Юниты лежат в `deploy/` (`rus-slam-server.service`, `rus-slam-display.service`, скрипт `display.sh`), порядок установки — `deploy/README.md`. Если робот без дисплея, второй сервис не нужен — страницы открывают с планшета.

Разовая проверка без `systemd`:

```
python3 gui/backend.py --kiosk      # сервер + страница на дисплее в браузере без рамок
python3 gui/backend.py --open       # сервер + страница в обычном окне браузера
python3 gui/backend.py              # только сервер (окон не открывает)
```

Что происходит на дисплее робота: данные обновляются каждые 300 мс, ввод PIN-кода выполняется там же, и отсек открывается **на самом роботе** — сервер вызывает команду замка (см. §3.1).

4.2. Инженерный пульт — удалённо

Пульт не запускают на работе: в него заходят с ноутбука или планшета в сети робота:

```
http://<ip-робота>:8080/console      инженерный пульт (адрес-синоним: /index.html)
```

IP печатается в консоли сервера при запуске (строка «инженерный пульт: ...» / «с планшета: ...»). Пульт видит те же данные и тот же замок, что и экран на работе: смена PIN и журнал доступа — общие.

4.3. Переход между экранами

Откуда → куда	Как
основной экран робота → инженерный пульт	ссылка « инженерный пульт → » в подвале экрана (на работе без дисплея — просто откройте <code>/console</code> на своём устройстве)
инженерный пульт → основной экран	кнопка « Основной экран » в шапке пульта или адрес <code>/</code>
напрямую	<code>/</code> — основной экран · <code>/index.html</code> , <code>/console</code> — пульт

4.4. С планшета или ноутбука — всё то же самое

Робот и устройство в одной сети (Wi-Fi цеха/стенда): `http://<ip-робота>:8080/` отдаёт основной экран любому браузеру, `.../console` — инженерный пульт. Так получатель груза может посмотреть состояние отсека со своего телефона, а инженер — работать с пультом, не подходя к роботу.

4.5. Остановка и перезапуск

- сервер: `Ctrl+C` в терминале, либо `sudo systemctl restart rus-slam-server;`
- страница на дисплее робота: `sudo systemctl restart rus-slam-display;`
- перезапуск не теряет данные: PIN и журнал доступа лежат в `gui/state/lock.json`, статистика пульта — в `localStorage` браузера.

5. Роли и защита доступа

Роль	Что может	Как ограничивается
Оператор смены	открыть/закрыть отсек, читать состояние	PIN-код отсека, 5 попыток → блокировка 30 с
Инженер-наладчик	ручные команды модулям, статистика, настройки	доступ к пульту по сети (см. §3.3), режим «стенд», интерлоки E-STOP
Получатель груза	только забрать груз	видит основной экран, но не может запустить движение

Что уже защищено технически:

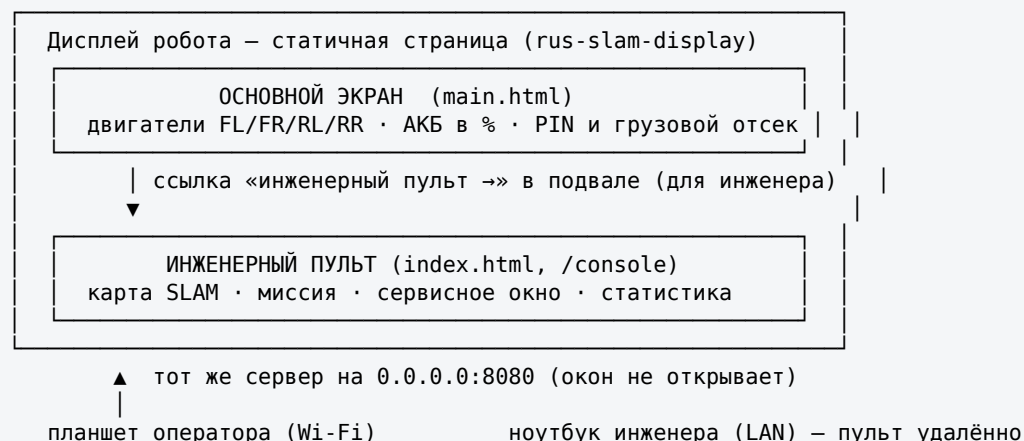
- PIN-код хранится как `SHA-256(salt + PIN)`, заводской 2580 меняется из сервисного окна или через `API POST /api/lock/pin` (см. §3);
- журнал доступа — с хеш-цепочкой: подделка или удаление записи обнаруживаются (`GET /api/audit` возвращает `integrity.ok = false`);
- открытие отсека в движении не предусмотрено: команды движения живут на инженерном пульте, а он в режиме миссии блокирует ручной ввод.

Развитие (документированное, не реализовано): вход в инженерный пульт по отдельному паролю инженера и/или ограничение доступа по IP, HTTPS, разграничение ролей «оператор/инженер/администратор», журнал входов.

6. Сценарии смены

Сценарий	Действия	Экран
Выдача груза получателю	робот приехал → получатель набирает PIN 2580 → «Открыть отсек» → забирает груз → «Закрыть отсек»	основной
Загрузка на складе	оператор открывает отсек по PIN, укладывает груз, закрывает; на пульте — «Подтвердить: груз взят»	основной + пульт
Контроль состояния	взгляд на экран: заряд в процентах, температура и состояние каждого модуля, связь	основной
Низкий заряд	при < 40 % подпись «низкий заряд», при < 33,5 В — «аварийный порог... движение запрещено»; оператор отправляет робота на базу	основной
Отладка модулей	инженер: пульт → сервисное окно → режим «стенд» → ручные команды FL/FR/RL/RR с контролем кадра UART	инженерный пульт
Разбор инцидента	пульт → сервисное окно → журнал доступа (аудит) и журнал сервисных действий; выгрузка CSV/JSON/печать отчёта	инженерный пульт
Демонстрация судьям	основной экран на дисплее робота: видно заряд, модули, отсек и время; смена PIN-кода с удалённого пульта на глазах у судей подтверждает защиту груза	основной + пульт

7. Раскладка на роботе



Один процесс backend.py = один источник данных для всех клиентов: сколько бы устройств ни открыли экраны, показания и состояние замка будут одинаковыми.

8. Быстрая проверка доступа

```

# 1. запустить бэкэнд (стенд, без робота)
python3 gui/backend.py --port 8080

# 2. убедиться, что API жив
curl -s http://127.0.0.1:8080/api/health
curl -s http://127.0.0.1:8080/api/state | head -c 300

# 3. отсек: неверный и верный PIN
curl -s -X POST http://127.0.0.1:8080/api/lock/open -H 'Content-Type: application/json' -d '{"pin":"1111"}'
curl -s -X POST http://127.0.0.1:8080/api/lock/open -H 'Content-Type: application/json' -d '{"pin":"2580"}'

# 4. проверить, что киоск отдаётся с корня
curl -sI http://127.0.0.1:8080/ | head -3
  
```

Автотесты доступа и API:

```
python3 gui/tests/backend.test.py # 40 проверок API и замка  
NODE_PATH=/tmp/harness/node_modules node gui/tests/main.api.test.js # 20: экран ↔ бэкенд
```

9. Лицензия и авторы

MIT, команда RUS SLAM, НТЦ АО «АВТОВАЗ», конкурс «Первые в профессии», задание № 2 «Автономное роботизированное устройство для внутренней логистики».